

Description Editorial Solutions

and 4.
Note that the five elements can be returned in any order.
It does not matter what you leave beyond the returned k (hence they underscores).

Constraints:

- $0 \leq \text{nums.length} \leq 100$
- $0 \leq \text{nums}[i] \leq 50$
- $0 \leq \text{val} \leq 100$



Seen this question in a real interview before? 1

Yes No

Accepted 5,483,372 / 8.8M

Acceptance Rate 62.2%

5.3K 980

Code

Java Auto

```
1 class Solution {
2     public int removeElement(int[] nums, int val) {
3
4         int k = 0;
5
6
7         for (int i = 0; i < nums.length; i++) {
8
9             if (nums[i] != val) {
10
11                 nums[k] = nums[i];
```

Saved Upgrade to Cloud Saving

Ln 3, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[3,2,2,3]

val =
3

Output

Description Editorial Solutions

26. Remove Duplicates from Sorted Array

Easy Topics Companies Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` is `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following code:

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length
```

```
int k = removeDuplicates(nums); // Calls your implementation
```

19.5K 1.1K

Code

Java Auto

```
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         int k = 1;
4
5         for (int i = 1; i < nums.length; i++) {
6             if (nums[i] != nums[i - 1]) {
7                 nums[k] = nums[i];
8                 k++;
9             }
10        }
11    }
```

Saved

Ln 15, Col 1

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[1,1,2]

Output

[1,2]

Expected

Description Accepted Editorial

26. Remove Duplicates from Sorted Array

Easy Topics Companies Hint

Given an integer array `nums` sorted in **non-decreasing order**, remove the duplicates **in-place** such that each unique element appears only **once**. The **relative order** of the elements should be kept the **same**.

Consider the number of *unique elements* in `nums` is `k`. After removing duplicates, return the number of unique elements `k`.

The first `k` elements of `nums` should contain the numbers in **sorted order**. The remaining elements beyond index `k - 1` can be ignored.

Custom Judge:

The judge will test your solution with the following

```
int[] nums = [...]; // Input array
int[] expectedNums = [...]; // The expected answer with correct length
```

```
int k = removeDuplicates(nums); // Calls your implementation
```

19.5K 1.1K

Code Code Sample

```
Java Auto
1 class Solution {
2     public int removeDuplicates(int[] nums) {
3         int k = 1;
4
5         for (int i = 1; i < nums.length; i++) {
6             if (nums[i] != nums[i - 1]) {
7                 nums[k] = nums[i];
8                 k++;
9             }
10        }
11    }
```

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

nums =
[1,1,2]

Output

[1,2]

Expected

Code

Medium Topics Companies

Example 1:

Output: 6

Example 2:

Output: 1

Example 3:

Output: 23

38.1K 571

Java Auto

Saved

Ln 13, Col 2

Testcase | **Test Result**

Accepted Runtime: 0 ms

✔ Case 1 ✔ Case 2 ✔ Case 3

Input

```
nums =  
[-2,1,-3,4,-1,2,1,-5,4]
```

Output

6

Expected

Description Accepted × Editorial

1732. Find the Highest Altitude

Easy Topics Companies Hint

There is a biker going on a road trip. The road trip consists of $n + 1$ points at various altitudes. The biker starts his trip on point 0 with altitude equal 0.

You are given an integer array `gain` of length n . `gain[i]` is the **net gain in altitude** between points i and $i + 1$ for all $(0 \leq i < n)$. Return the **highest altitude** of a point.

Example 1:

Input: `gain = [-5,1,5,0,-7]`
Output: 1
Explanation: The altitudes are `[0,-5,-4,1,1,-6]`. The highest is 1.

Example 2:

Input: `gain = [-4,-3,-2,-1,4,3,2]`
Output: 0
Explanation: The altitudes are `[0,-4,-7,-9,-8,-4,-1,1]`. The highest is 0.

3.5K 426 ☆ 🔗 ?

<https://leetcode.com>

Code

Java Auto

```
1 class Solution {
2     public int largestAltitude(int[] gain) {
3         int maxAltitude = 0;
4         int currentAltitude = 0;
5
6         for (int i = 0; i < gain.length; i++) {
7             currentAltitude += gain[i];
8             maxAltitude = Math.max(maxAltitude, currentAltitude);
9         }
10
11         return maxAltitude;
12     }
13 }
```

Saved

Ln 13, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

gain =
[-5,1,5,0,-7]

Output

1

Expected

Problem List

49. Group Anagrams

Medium

Topics

Companies

Given an array of strings `strs`, group the anagrams together. You can return the answer in **any order**.

Example 1:

Input: `strs = ["eat","tea","tan","ate","nat","bat"]`

Output: `[["bat"],["nat","tan"],["ate","eat","tea"]]`

Explanation:

- There is no string in `strs` that can be rearranged to form `"bat"`.
- The strings `"nat"` and `"tan"` are anagrams; they can be rearranged to form each other.
- The strings `"ate"`, `"eat"`, and `"tea"` are anagrams as they can be rearranged to form each other.

Save

22.3K 422

Code

Java

Auto

Ln 15, Col 2

```
6 char[] chars = str.toCharArray();
7 Arrays.sort(chars);
8 String key = new String(chars);
9
10 map.computeIfAbsent(key, k -> new ArrayList<>()).add(str);
11
12
13 return new ArrayList<>(map.values());
14
15 }
```

Accepted

Runtime: 1 ms

Case 1

Case 2

Case 3

Input

strs = ["eat","tea","tan","ate","nat","bat"]

Output

[["eat","tea","ate"],["bat"],["tan","nat"]]

Expected

31°C Partly sunny

Search

ENG IN

9:06 AM 7/29/2026

347. Top K Frequent Elements

Medium Topics Companies

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements. You may return the answer in any order.

Example 1:

Input: `nums = [1,1,1,2,2,3]`, `k = 2`

Output: `[1,2]`

Example 2:

Input: `nums = [1]`, `k = 1`

Output: `[1]`

Example 3:

Input: `nums = [1,2,1,2,1,2,3,1,3,2]`,

Output: `[1,2]`

19.8K 494 ☆ 🔗 ⓘ

Code

Java Auto

```
24         if (counter == k) {
25             return res;
26         }
27     }
28 }
29
30 return res;
31
32 }
33 }
```

Saved

Ln 33, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

nums =
[1,1,1,2,2,3]

k =
2

Output



In computer science, a double-ended queue (dequeue, often abbreviated to deque, pronounced deck) is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail).

Deque interfaces can be implemented using various types of collections such as `LinkedList` or `ArrayDeque` classes. For example, deque can be declared as:

```
Deque deque = new LinkedList<>();  
or  
Deque deque = new ArrayDeque<>();
```

You can find more details about Deque [here](#).

In this problem, you are given N integers. You need to find the maximum number of unique integers among all the possible contiguous subarrays of size M .

Note: Time limit is 3 second for this problem.

Input Format

The first line of input contains two integers N and M : representing the total number of integers and the size of the subarray, respectively. The next line contains N space separated integers.

Constraints

$$1 \leq N \leq 100000$$

$$1 \leq M \leq 100000$$

$$M \leq N$$

The numbers in the array will range between $[0, 10000000]$.

```
28         map.put(first, count - 1);  
29     }  
30 }  
31 }  
32  
33     System.out.println(maxUnique);  
34     in.close();  
35 }  
36 }  
37
```

Line: 36 Col: 2

[Upload Code as File](#)☐ Test against custom input[Run Code](#)[Submit Code](#)

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Input (stdin)

```
1 6 3  
2 5 3 5 2 3 2
```

[Download](#)

Your Output (stdout)

```
1 3
```


In computer science, a set is an abstract data type that can store certain values, without any particular order, and no repeated values(Wikipedia). $\{1, 2, 3\}$ is an example of a set, but $\{1, 2, 2\}$ is not a set. Today you will learn how to use sets in java by solving this problem.

You are given n pairs of strings. Two pairs (a, b) and (c, d) are identical if $a = c$ and $b = d$. That also implies (a, b) is not same as (b, a) . After taking each pair as input, you need to print number of unique pairs you currently have.

Complete the code in the editor to solve this problem.

Input Format

In the first line, there will be an integer T denoting number of pairs. Each of the next T lines will contain two strings separated by a single space.

Constraints:

- $1 \leq T \leq 100000$
- Length of each string is atmost 5 and will consist lower case letters only.

Output Format

Print T lines. In the i_{th} line, print number of unique pairs you have after taking i^{th} pair as input.

Sample Input

```
5
john tom
john mary
```

```
11 int t = s.nextInt();
12 String [] pair_left = new String[t];
13 String [] pair_right = new String[t];
14
15 for (int i = 0; i < t; i++) {
16     pair_left[i] = s.next();
17     pair_right[i] = s.next();
18 }
19
20 HashSet<String> set = new HashSet<>();
21
22 for (int i = 0; i < t; i++) {
23     set.add(pair_left[i] + " " + pair_right[i]);
24     System.out.println(set.size());
25 }
26
27 s.close();
```

Line: 26 Col: 19

[Upload Code as File](#)☐ Test against custom input[Run Code](#)[Submit Code](#)

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

[Sample Test case 0](#)

Input (stdin)

```
1 5
```

[Download](#)